EECE 2323 Digital Logic Design Lab

Name: Arefin Mahmood **Lab Report 5: Adding Instruction Decoding to the Datapath**
Date: 06/08/2026

Background & Purpose

In this lab, I added instruction decoding to the datapath from Lab 4. In the previous lab, the datapath was controlled mostly through manual VIO inputs. In this lab, the VIO gives instruction fields such as rs1, rs2, rd, immediate, nzimm, offset, and opcode. The instruction decoder uses those fields to create the datapath control signals. These control signals decide which registers are read or written, which ALU operation runs, whether data memory is written, and whether data memory or the ALU result is written back to the register file. This lab is important because real processors do not manually set every control signal. A processor uses instruction decoding so the opcode controls the datapath automatically.

Design Analysis

The main change from Lab 4 was the addition of the instruction decoder. Instead of directly controlling signals like RegWrite, ALUSrc2, ALUOp, MemWrite, and MemToReg from VIO, the VIO now sends instruction fields into the decoder. The decoder is combinational logic, so it does not need a clock. Its outputs depend only on the current opcode and immediate values. The clock is still needed for writing to the register file and writing to data memory.

The final design includes the register file, ALU, data memory, instruction decoder, debounce circuit, VIO, and several muxes. The muxes select the correct register addresses, ALU inputs, and writeback source. For example, in an addi instruction, the decoder sets ALUSrc2 to use the sign-extended nzimm value as the second ALU input. In an lw instruction, the decoder sets MemToReg so the data memory output is written back to the register file.

Results and Analysis

I first tested the instruction decoder in simulation. The testbench changed the opcode from 0000 through 1011 and also tested an invalid opcode. The waveform in Figure A1 shows that each opcode produced the expected control outputs. For example, lw enabled RegWrite and MemToReg, sw enabled MemWrite, ALU instructions enabled RegWrite, and branch instructions changed ALUOp to the branch operation. This confirmed that the decoder logic worked before programming the FPGA.

After simulation, I tested the full datapath using the VIO dashboard. First, I used addi to create 16'h0005 and write it into register 2, as shown in Figure C1. Then I wrote 16'h0003 into register 1 and used the add instruction to add register 2 and register 1. Figure C2 shows register 2 holding 16'h0005, register 1 holding 16'h0003, and the ALU output equal to 16'h0008. Next, I tested andi by using register 3 with immediate 16'h0007. Figure C3 shows that 16'h0008 AND 16'h0007 produced 16'h0000. I also tested branch instructions. Figure C4 shows beqz setting alu_take_branch to 1 when the selected register was zero. Figure C5 shows bneqz setting alu_take_branch to 1 when register 3 contained 16'h0008. Finally, I tested memory by storing 16'h0008 into data memory and loading it into register 5. Figure C7 shows register 5 containing 16'h0008 after the load.

Conclusion & Recommendations

This lab showed how instruction decoding controls a datapath. I learned how the opcode creates the control signals needed for the register file, ALU, memory, and muxes. The simulation confirmed that the decoder produced the correct outputs for each opcode, and the hardware VIO tests confirmed that the full datapath worked for addi, add, andi, branch, store, and load instructions. For future testing, I would recommend testing each instruction in two steps: first check the ALU or memory output before pressing the clock button, then clear the active instruction and read back the destination register. This avoids confusion from the live datapath recalculating values while checking saved register contents..

Appendix A: Prelab and Test Data/Simulation

The instruction decoder is combinational logic. It takes the opcode and immediate fields from VIO and creates the control signals for the datapath. For Tables A1 and A2, X means don't care.

For this design:
RegDst: 0 = rd, 1 = rs2
Regsrc: 0 = rs1, 1 = rd

Table A1: Instruction Decoder Inputs

Operation	Opcode	Immediate	Offset	nzimm
lw	0000	imm	X	X
sw	0001	imm	X	X
add	0010	X	X	X
addi	0011	X	X	nzimm
and	0100	X	X	X
andi	0101	imm	X	X
or	0110	X	X	X
xor	0111	X	X	X
srai	1000	X	X	nzimm
slli	1001	X	X	nzimm
beqz	1010	X	offset	X
bneqz	1011	X	offset	X

Table A2: Instruction Decoder Outputs

Operation	RegWrite	RegDst	instr_i	ALUSrc1	ALUSrc2	ALUOp	MemWrite	MemToReg	Regsrc
lw	1	1	imm	0	1	0000	0	1	0
sw	0	X	imm	0	1	0000	1	X	0
add	1	0	X	0	0	0000	0	0	1
addi	1	0	nzimm	0	1	0000	0	0	1
and	1	0	X	0	0	0010	0	0	1
andi	1	0	imm	0	1	0010	0	0	1
or	1	0	X	0	0	0011	0	0	1
xor	1	0	X	0	0	1000	0	0	1
srai	1	0	nzimm	0	1	0100	0	0	1

slli	1	0	nzimm	0	1	0101	0	0	1
beqz	0	X	offset	0	X	0110	0	X	0
bneqz	0	X	offset	0	X	0111	0	X	0

A.2 Test Data and Simulation

The instruction decoder was tested with a SystemVerilog testbench. The testbench kept the immediate fields constant and changed the opcode every 100 ns. This tested every instruction type from the prelab table.

Test values used:
immediate = 7'b0000101
nzimm = 6'b000011
offset = 9'b000000100

Table A3: Instruction Decoder Simulation Test Plan

Test	Opcode	Instruction	Expected Result
1	0000	lw	RegWrite=1, MemToReg=1, ALUSrc2=1, ALUOp=0000
2	0001	sw	RegWrite=0, MemWrite=1, ALUSrc2=1, ALUOp=0000
3	0010	add	RegWrite=1, ALUSrc2=0, ALUOp=0000
4	0011	addi	RegWrite=1, ALUSrc2=1, ALUOp=0000
5	0100	and	RegWrite=1, ALUSrc2=0, ALUOp=0010
6	0101	andi	RegWrite=1, ALUSrc2=1, ALUOp=0010
7	0110	or	RegWrite=1, ALUSrc2=0, ALUOp=0011
8	0111	xor	RegWrite=1, ALUSrc2=0, ALUOp=1000
9	1000	srai	RegWrite=1, ALUSrc2=1, ALUOp=0100
10	1001	slli	RegWrite=1, ALUSrc2=1, ALUOp=0101
11	1010	beqz	RegWrite=0, ALUOp=0110
12	1011	bneqz	RegWrite=0, ALUOp=0111
13	1111	invalid	Main control signals return to 0

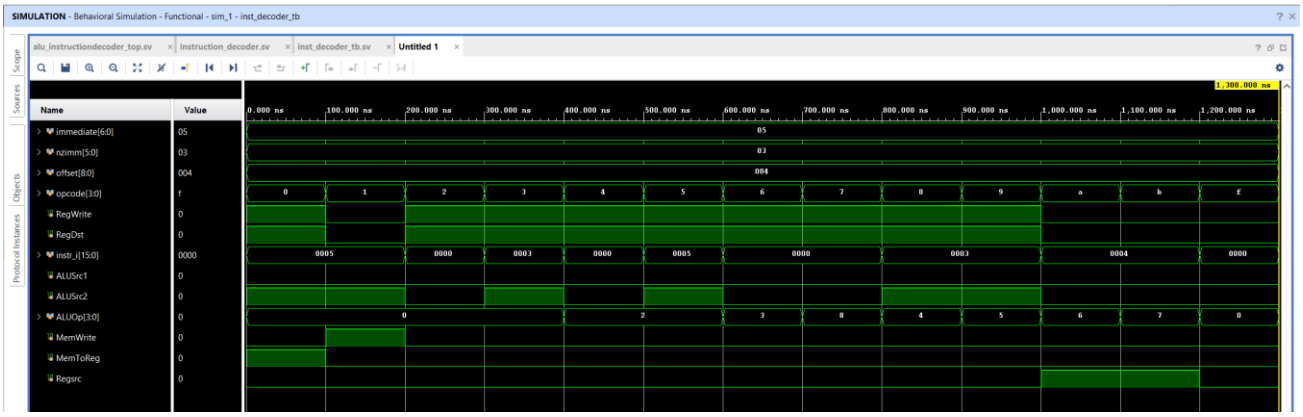
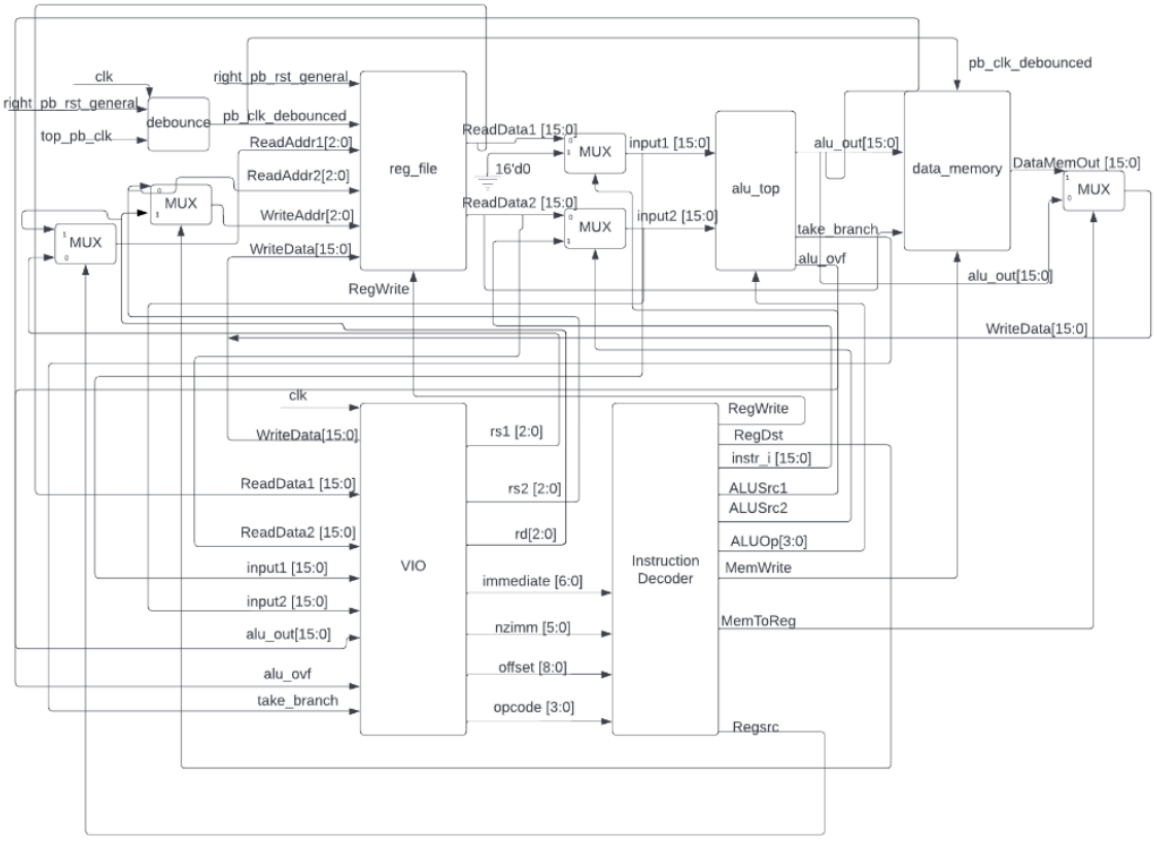


Figure A1: Instruction Decoder Simulation Waveform

The waveform shows the instruction decoder outputs changing as the opcode changes from 0000 through 1011, plus an invalid opcode. This confirms that the decoder generates the expected control signals for load, store, ALU, immediate, and branch instructions.

Appendix B: Design Figures

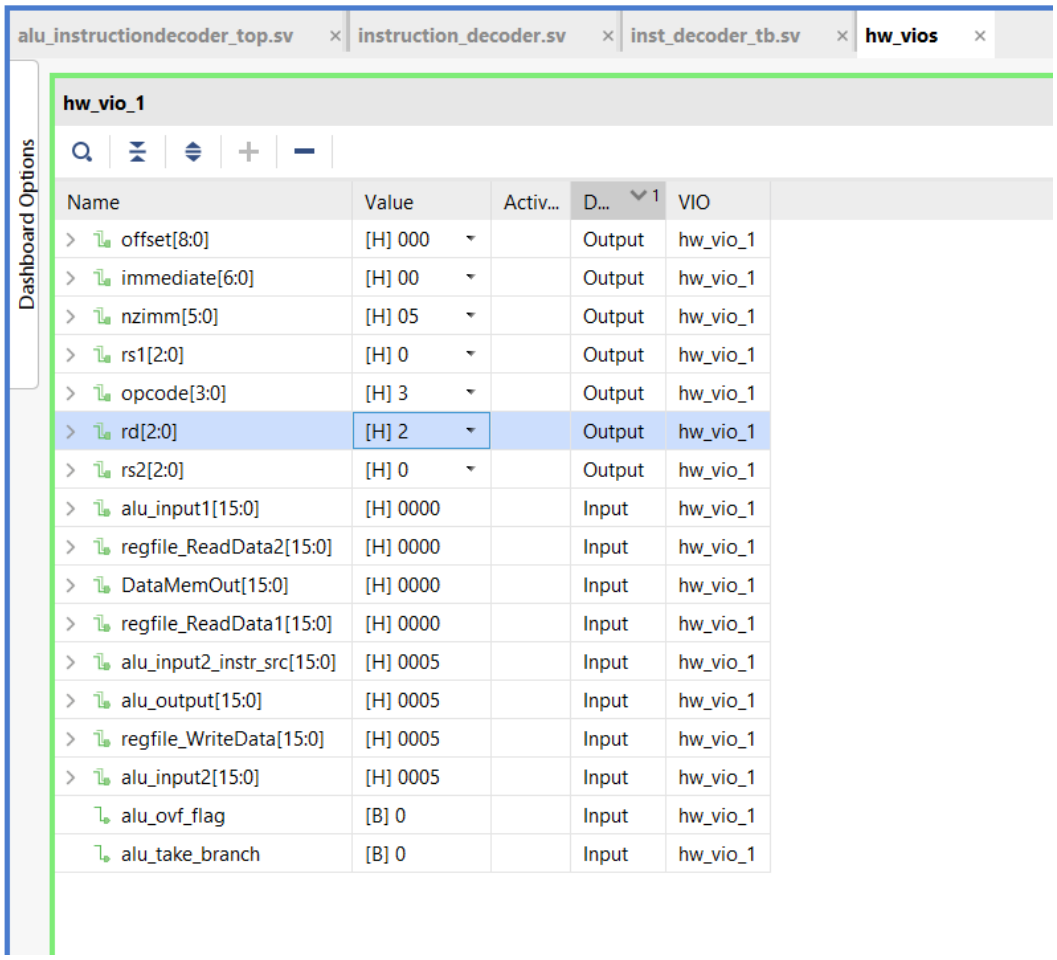
Figure B: Lab 5 datapath with instruction decoder



This diagram shows the final Lab 5 datapath. VIO provides instruction fields to the instruction decoder. The decoder sends control signals to the muxes, register file, ALU, and data memory.

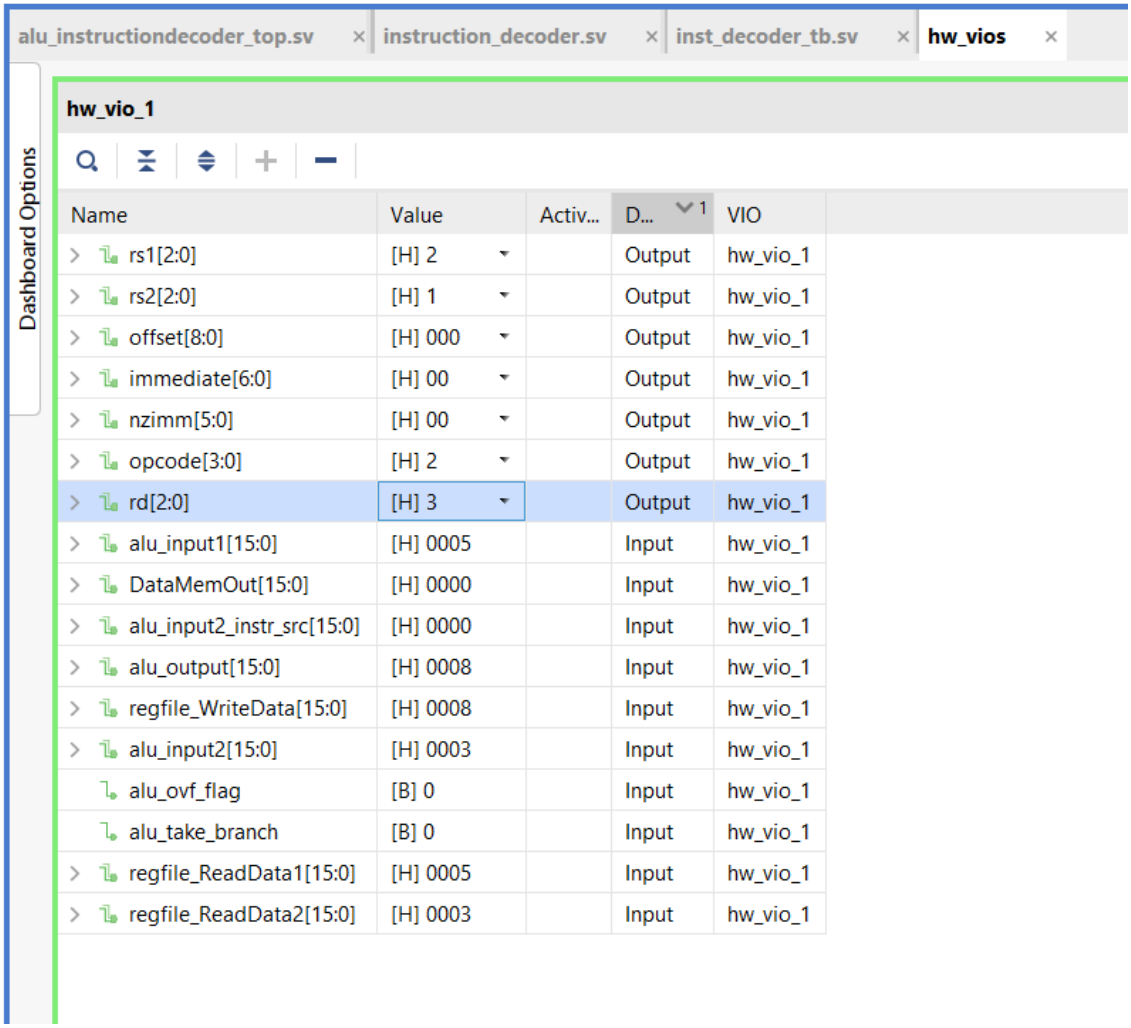
Appendix C: Hardware Evidence

Figure C1: ADDI instruction creating 16'h0005 for register 2



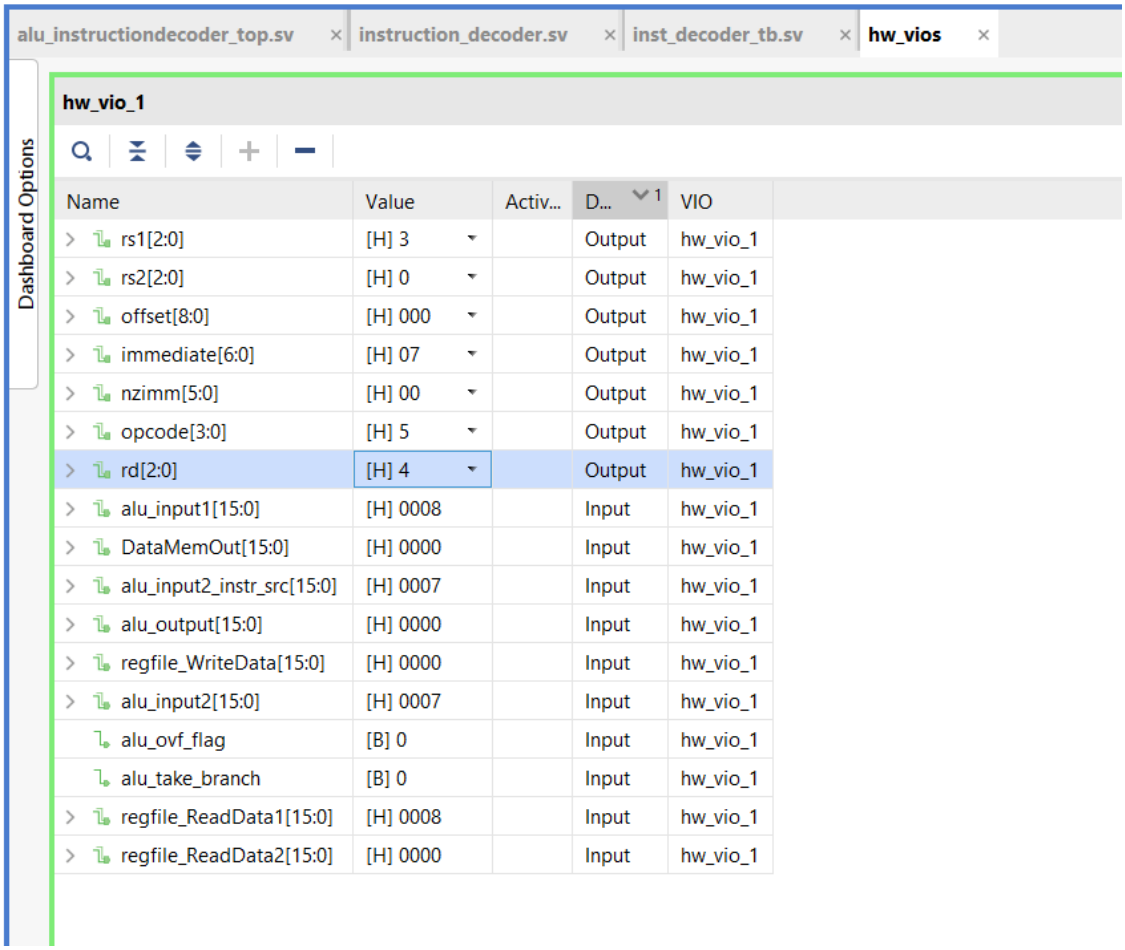
This VIO screenshot shows opcode 3 for addi, nzimm set to 05, and rd set to 2. The ALU output and regfile_WriteData are both 16'h0005.

Figure C2: ADD instruction adding register 2 and register 1



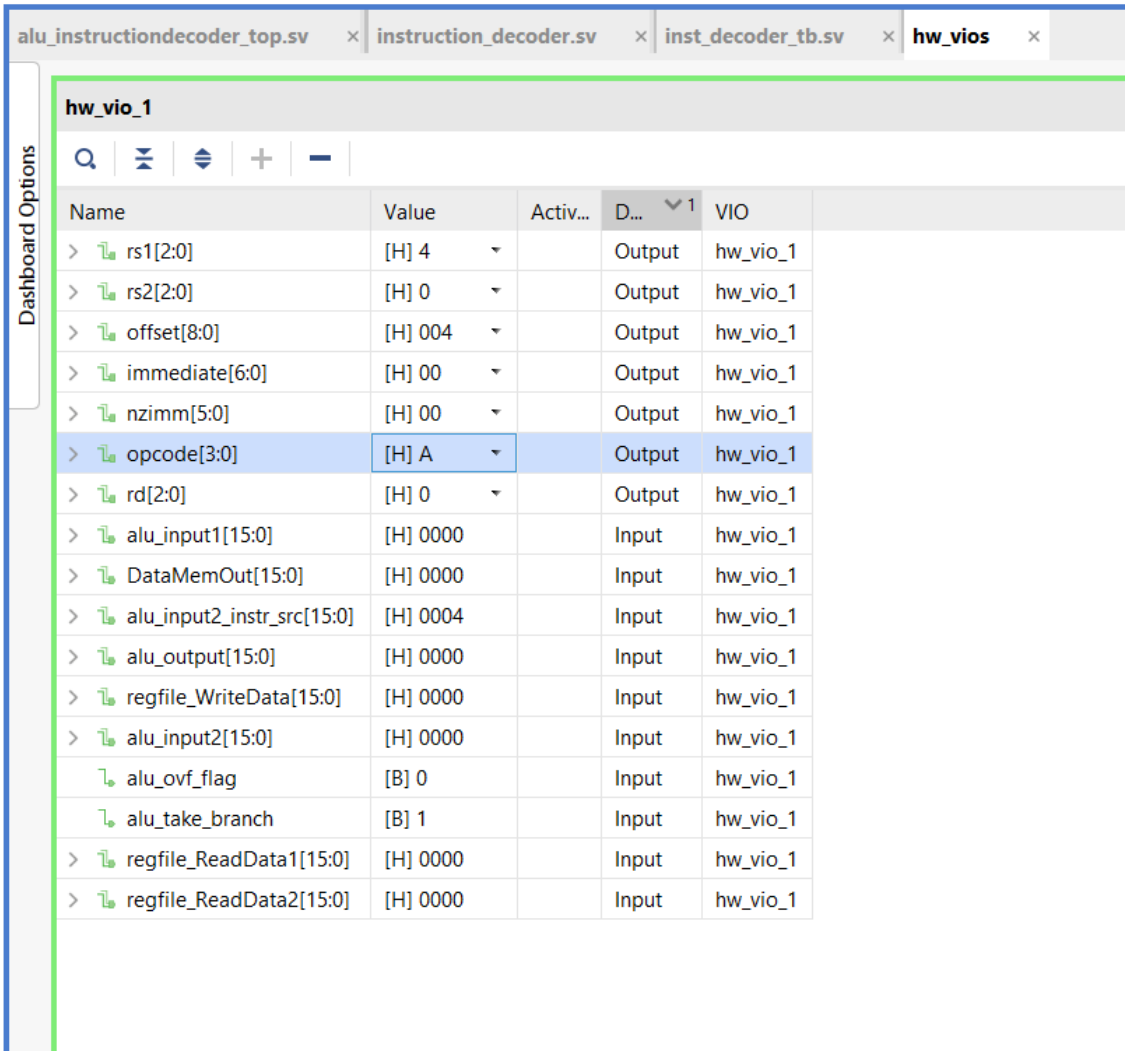
This VIO screenshot shows rs1 set to 2 and rs2 set to 1. Register 2 contains 16'h0005 and register 1 contains 16'h0003. The ALU output is 16'h0008.

Figure C3: ANDI instruction using register 3 and immediate 16'h0007



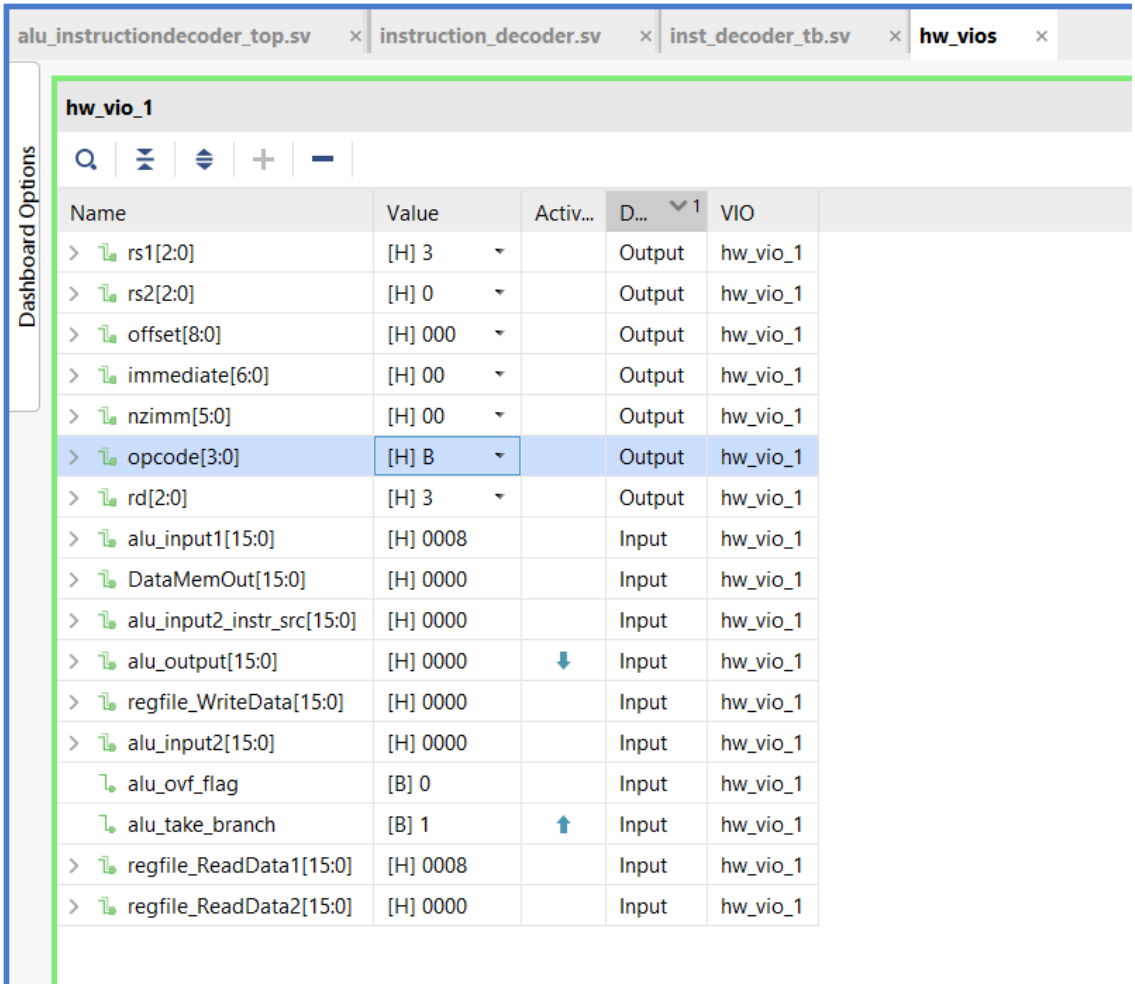
This VIO screenshot shows register 3 containing 16'h0008 and immediate set to 07. The ALU output is 16'h0000 because 16'h0008 AND 16'h0007 equals 16'h0000.

Figure C4: BEQZ branch test



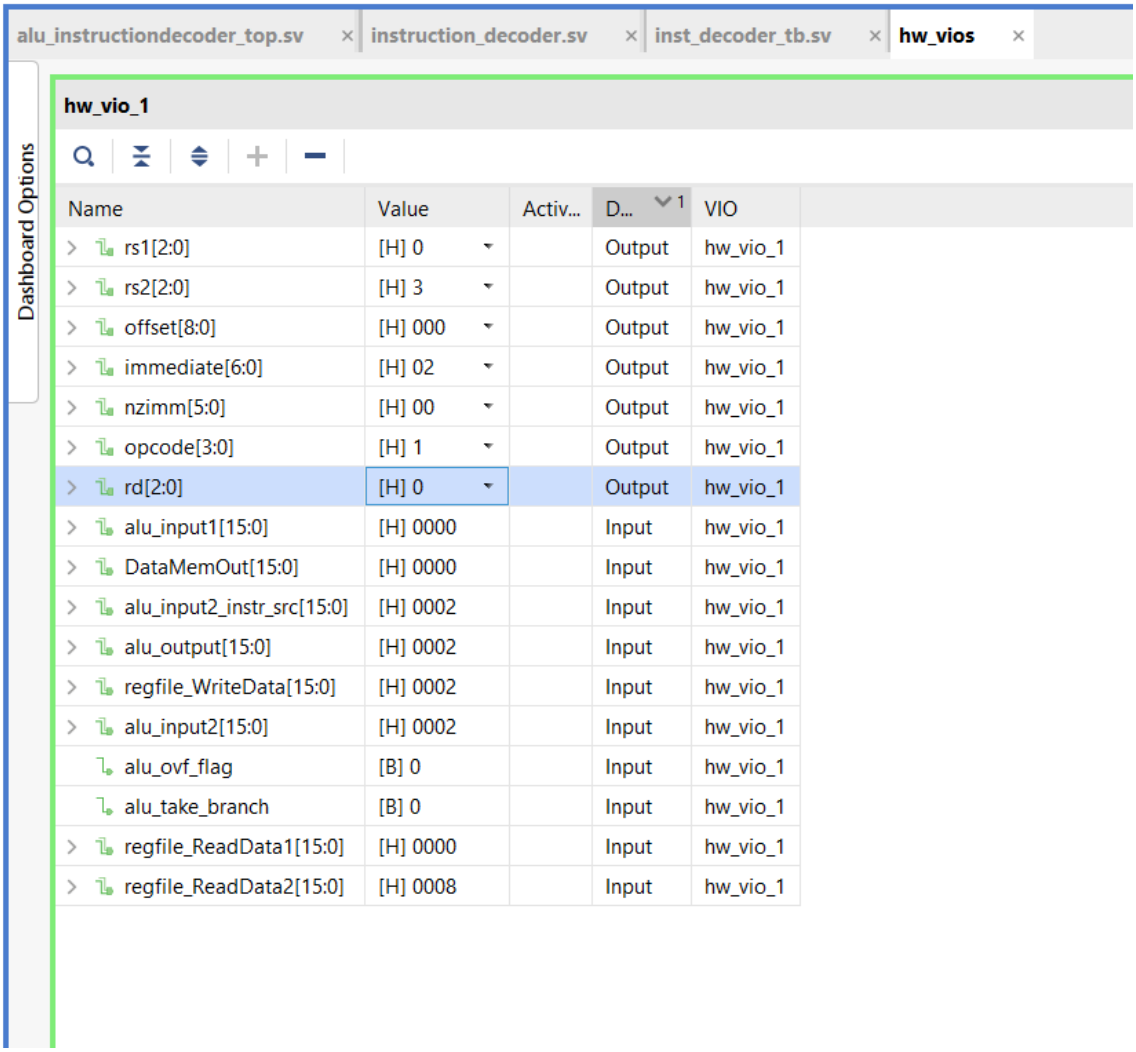
This VIO screenshot shows opcode A for beqz. The selected register value is 16'h0000, so alu_take_branch is 1.

Figure C5: BNEQZ branch test



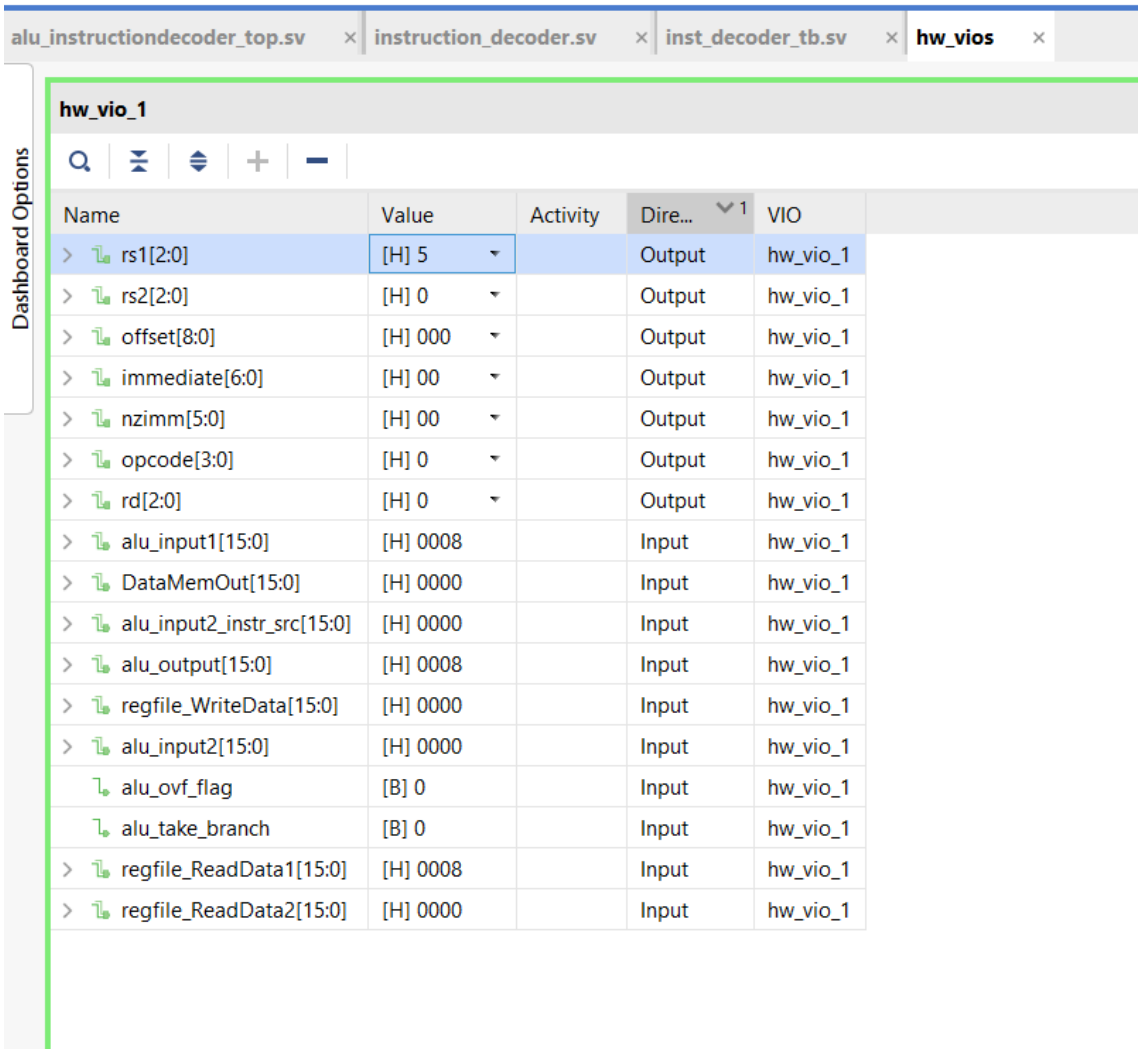
This VIO screenshot shows opcode B for bneqz. Register 3 contains 16'h0008, so alu_take_branch is 1.

Figure C6: SW instruction storing register 3 into memory address 2



This VIO screenshot shows opcode 1 for sw, rs2 set to 3, and immediate set to 02. Register 3 contains 16'h0008, and the memory address is calculated through the ALU.

Figure C7: LW result checked in register 5



This VIO screenshot verifies that the value 16'h0008 was loaded from data memory and written into register 5. With rs1 set to 5, regfile_ReadData1 shows 16'h0008.

Appendix D: Design Program Files

The key files written for this lab are the instruction decoder and the instruction decoder testbench. Other project files used in the Vivado design include alu_instructiondecoder_top.sv, regfile.sv, alu.sv, TwoToOneMux.sv, debounce.sv, adaptor_display.sv, data_memory IP, and vio_0 IP.

Appendix D.1: instruction_decoder.sv

```
module instruction_decoder (
    // input fields from vio / mapper
    input  logic [6:0] immediate,
    input  logic [5:0] nzimm,
    input  logic [8:0] offset,
    input  logic [3:0] opcode,

    // output control signals for the datapath
    output logic RegWrite,
    output logic RegDst,
    output logic [15:0] instr_i,
    output logic ALUSrc1,
    output logic ALUSrc2,
    output logic [3:0] ALUOp,
    output logic MemWrite,
    output logic MemToReg,
    output logic Regsrc
);

// combinational decoder logic
always_comb begin

    // default values so nothing random happens
    RegWrite = 1'b0;
    RegDst   = 1'b0;
    instr_i  = 16'h0000;
    ALUSrc1  = 1'b0;
    ALUSrc2  = 1'b0;
    ALUOp    = 4'b0000;
    MemWrite = 1'b0;
    MemToReg = 1'b0;
    Regsrc   = 1'b0;

    // choose control signals based on opcode
    case (opcode)

        // lw: load word from data memory into register
        4'b0000: begin
            RegWrite = 1'b1;
            RegDst   = 1'b1;
            instr_i  = {{9{immediate[6]}}, immediate};
            ALUSrc1  = 1'b0;
            ALUSrc2  = 1'b1;
            ALUOp    = 4'b0000;
            MemWrite = 1'b0;
```

```
    MemToReg = 1'b1;
    Regsrc   = 1'b0;
end

// sw: store register value into data memory
4'b0001: begin
    RegWrite = 1'b0;
    instr_i  = {{9{immediate[6]}}, immediate};
    ALUSrc1  = 1'b0;
    ALUSrc2  = 1'b1;
    ALUOp    = 4'b0000;
    MemWrite = 1'b1;
    MemToReg = 1'b0;
    Regsrc   = 1'b0;
end

// add: add two register values
4'b0010: begin
    RegWrite = 1'b1;
    RegDst   = 1'b1;
    ALUSrc1  = 1'b0;
    ALUSrc2  = 1'b0;
    ALUOp    = 4'b0000;
    MemWrite = 1'b0;
    MemToReg = 1'b0;
    Regsrc   = 1'b0;
end

// addi: add register value with immediate
4'b0011: begin
    RegWrite = 1'b1;
    RegDst   = 1'b1;
    instr_i  = {{10{nzimm[5]}}, nzimm};
    ALUSrc1  = 1'b0;
    ALUSrc2  = 1'b1;
    ALUOp    = 4'b0000;
    MemWrite = 1'b0;
    MemToReg = 1'b0;
    Regsrc   = 1'b0;
end

// and: bitwise and using two registers
4'b0100: begin
    RegWrite = 1'b1;
    RegDst   = 1'b1;
    ALUSrc1  = 1'b0;
    ALUSrc2  = 1'b0;
    ALUOp    = 4'b0010;
    MemWrite = 1'b0;
    MemToReg = 1'b0;
    Regsrc   = 1'b0;
end
```

```
// andi: bitwise and using register and immediate
4'b0101: begin
   RegWrite = 1'b1;
   RegDst   = 1'b1;
   instr_i  = {{9{immediate[6]}}, immediate};
   ALUSrc1  = 1'b0;
   ALUSrc2  = 1'b1;
   ALUOp    = 4'b0010;
   MemWrite = 1'b0;
   MemToReg = 1'b0;
   Regsrc   = 1'b0;
end

// or: bitwise or using two registers
4'b0110: begin
   RegWrite = 1'b1;
   RegDst   = 1'b1;
   ALUSrc1  = 1'b0;
   ALUSrc2  = 1'b0;
   ALUOp    = 4'b0011;
   MemWrite = 1'b0;
   MemToReg = 1'b0;
   Regsrc   = 1'b0;
end

// xor: bitwise xor using two registers
4'b0111: begin
   RegWrite = 1'b1;
   RegDst   = 1'b1;
   ALUSrc1  = 1'b0;
   ALUSrc2  = 1'b0;
   ALUOp    = 4'b1000;
   MemWrite = 1'b0;
   MemToReg = 1'b0;
   Regsrc   = 1'b0;
end

// srai: arithmetic shift right immediate
4'b1000: begin
   RegWrite = 1'b1;
   RegDst   = 1'b1;
   instr_i  = {{10{nzimm[5]}}, nzimm};
   ALUSrc1  = 1'b0;
   ALUSrc2  = 1'b1;
   ALUOp    = 4'b0100;
   MemWrite = 1'b0;
   MemToReg = 1'b0;
   Regsrc   = 1'b0;
end

// slli: shift left logical immediate
4'b1001: begin
   RegWrite = 1'b1;
```

```
            RegDst   = 1'b1;
            instr_i  = {{10{nzimm[5]}}, nzimm};
            ALUSrc1  = 1'b0;
            ALUSrc2  = 1'b1;
            ALUOp    = 4'b0101;
            MemWrite = 1'b0;
            MemToReg = 1'b0;
            Regsrc   = 1'b0;
        end

        // beqz: branch if value equals zero
        4'b1010: begin
            RegWrite = 1'b0;
            instr_i  = {{7{offset[8]}}, offset};
            ALUSrc1  = 1'b0;
            ALUSrc2  = 1'b0;
            ALUOp    = 4'b0110;
            MemWrite = 1'b0;
            MemToReg = 1'b0;
            Regsrc   = 1'b1;
        end

        // bneqz: branch if value does not equal zero
        4'b1011: begin
            RegWrite = 1'b0;
            instr_i  = {{7{offset[8]}}, offset};
            ALUSrc1  = 1'b0;
            ALUSrc2  = 1'b0;
            ALUOp    = 4'b0111;
            MemWrite = 1'b0;
            MemToReg = 1'b0;
            Regsrc   = 1'b1;
        end

        // default: invalid opcode, no write happens
        default: begin
            RegWrite = 1'b0;
            RegDst   = 1'b0;
            instr_i  = 16'h0000;
            ALUSrc1  = 1'b0;
            ALUSrc2  = 1'b0;
            ALUOp    = 4'b0000;
            MemWrite = 1'b0;
            MemToReg = 1'b0;
            Regsrc   = 1'b0;
        end

    endcase
end

endmodule
```

Appendix D.2: inst_decoder_tb.sv

```
`timescale 1ns / 1ps

module inst_decoder_tb();

// input signals going into the decoder
logic [6:0] immediate;
logic [5:0] nzimm;
logic [8:0] offset;
logic [3:0] opcode;

// output signals coming from the decoder
logic RegWrite;
logic RegDst;
logic [15:0] instr_i;
logic ALUSrc1;
logic ALUSrc2;
logic [3:0] ALUOp;
logic MemWrite;
logic MemToReg;
logic Regsrc;

// connect the testbench signals to the instruction decoder
instruction_decoder UUT (
    .immediate(immediate),
    .nzimm(nzimm),
    .offset(offset),
    .opcode(opcode),
    .RegWrite(RegWrite),
    .RegDst(RegDst),
    .instr_i(instr_i),
    .ALUSrc1(ALUSrc1),
    .ALUSrc2(ALUSrc2),
    .ALUOp(ALUOp),
    .MemWrite(MemWrite),
    .MemToReg(MemToReg),
    .Regsrc(Regsrc)
);

// start the test
initial begin

    // test values for the immediate fields
    immediate = 7'b0000101;
    nzimm     = 6'b000011;
    offset    = 9'b000000100;

    // first opcode starts at time 0
    opcode = 4'b0000; // lw

    // test every opcode from the lab 5 prelab table
    #100 opcode = 4'b0001; // sw
    #100 opcode = 4'b0010; // add
```

```
    #100 opcode = 4'b0011; // addi
    #100 opcode = 4'b0100; // and
    #100 opcode = 4'b0101; // andi
    #100 opcode = 4'b0110; // or
    #100 opcode = 4'b0111; // xor
    #100 opcode = 4'b1000; // srai
    #100 opcode = 4'b1001; // slli
    #100 opcode = 4'b1010; // beqz
    #100 opcode = 4'b1011; // bneqz

    // test invalid opcode to make sure default values work
    #100 opcode = 4'b1111; // invalid

    // wait before ending simulation
    #100 $finish;
end

endmodule
```